

Birkbeck

(University of London)

M.Sc. Coursework

School of Computing and Mathematical Sciences

Natural Language Processing (COIY064H7)

Question:	1	2	3	Total
Points:	30	45	25	100

- For this module, 60% of the overall marks are for this programming assignment and 40% are for an in-class test.
- The coursework consists of a portfolio of programming tasks with three parts marked out of a total of 100.
- You must complete the coursework using the Python 3 programming language. You may submit an `.ipynb` notebook or a regular python script.
- In some parts, the questions specify that you must implement something yourself, or use a particular python library, but otherwise you may use any widely available python libraries.
- Most of the question parts only require code to answer. For a few parts you are asked additional questions requiring brief text answers, you may add these either as comments in your code, or in a separate text file.
- For the classification questions, marks are awarded for demonstrating full understanding of the methods, not for achieving the best possible accuracy or F1 score.
- If a part of a question is unspecified (e.g., a pre-processing step), you should make reasonable choices and explain these choices in comments in your code or text answers.
- Some marks are allocated for a local git commit history which you should include within the zip file that you upload to Moodle as your submission. I will demonstrate how to do this in class.

1. Part One — Syntax and Style

In the first part of your coursework, your task is to explore the syntax and style of a set of 19th Century novels using the methods and tools that you learned in class.

The texts you need for this part are in the `texts` directory in the coursework Moodle template. The texts are in plain text files, and the filenames include the title, author, and year of publication, separated by hyphens. The template code provided in `PartOne.py` includes function headers for some sub-parts of this question. Your finished script should call each of these functions in order. To complete your coursework, complete these functions so that they perform the tasks specified in the questions below. You may (and in some cases should) define additional functions.

- (a) `read_novels`: Each file in the `texts` directory contains the text of a novel, and the name of the file is the title, author, and year of publication of the novel, separated by hyphens. Write a python function `read_novels` to do the following:
 - i. create a pandas dataframe with the following columns: text, title, author, year
 - ii. sort the dataframe by the year column before returning it, resetting or ignoring the dataframe index.3
- (b) `nltk_ttr`: This function should return a dictionary mapping the title of each novel to its type-token ratio. Tokenize the text using the NLTK library only. Do not include punctuation as tokens, and ignore case when counting types.3
- (c) `flesch_kincaid`: This function should return a dictionary mapping the title of each novel to the Flesch-Kincaid reading ease score of the text. Use the NLTK library for tokenization and the CMU pronouncing dictionary for estimating syllable counts.4
- (d) `parse`: The goal of this function is to process the texts with spaCy's tokenizer and parser, and store the processed texts. Your completed function should:
 - i. Use the spaCy `nlp` method to add a new column to the dataframe that contains parsed and tokenized Doc objects for each text.
 - ii. Serialise the resulting dataframe (i.e., write it out to disk) using the pickle format.
 - iii. Return the dataframe.
 - iv. Load the dataframe from the pickle file and use it for the remainder of this coursework part. Note: one or more of the texts may exceed the default maximum length for spaCy's model. You will need to either increase this length or parse the text in sections.6
- (e) Working with parses: the final lines of the code template contain three `for` loops. Write the functions needed to complete these loops so that they print:
 - i. The title of each novel and a list of the ten most common syntactic subjects overall in the text.3
 - ii. The title of each novel and a list of the verbs most likely to occur with the subject 'he', ordered by their Pointwise Mutual Information.3

- iii. The title of each novel and a list of the verbs most likely to occur with the subject 'she', ordered by their Pointwise Mutual Information. 3
- (f) Five marks are allocated for your git commit history. You should make regular, atomic commits with concise but informative commit messages. See the section titled **Submission** for more details. 5

Part One total marks: 30

2. Part Two — Feature Extraction and Classification

In the second part of the coursework, your task is to train and test machine learning classifiers on a dataset of political speeches. The objective is to learn to predict the political party from the text of the speech.

Your final script should print out the answers to each part where required.

- (a) Read the `hansard10000.csv` dataset in the `texts` directory into a dataframe. Subset and rename the dataframe as follows: 8
 - i. rename the 'Labour (Co-op)' value in 'party' column to 'Labour', and then:
 - ii. remove any rows where the value of the 'party' column is not one of the four most common party names, and remove the 'Speaker' value.
 - iii. remove any rows where the value in the 'speech_class' column is not 'Speech'.
 - iv. remove any rows where the text in the 'speech' column is less than 1000 characters long.

Print the dimensions of the resulting dataframe using the shape method.

- (b) Vectorise the speeches using `TfidfVectorizer` from `scikit-learn`. Use the default parameters, except for omitting English stopwords and setting `max_features` to 3000. Split the data into a train and test set, using stratified sampling, with a random seed of 26. Then, train `RandomForest` (with `n_estimators=300`) and `SVM` with linear kernel classifiers on the training set, and print the `scikit-learn` macro-average f1 score and classification report for each classifier on the test set. The label that you are trying to predict is the 'party' value. 10
- (c) Adjust the parameters of the `TfidfVectorizer` so that unigrams, bi-grams and tri-grams will be considered as features, limiting the total number of features to 3000. Print the classification report as in 2(c) again using these parameters. 5
- (d) Implement a new custom tokenizer and pass it to the `tokenizer` argument of `TfidfVectorizer`. You can use this function in any way you like to try to achieve the best classification performance while keeping the number of features to no more than 3000, and using the same three classifiers as above. Print the classification report for the best performing classifier using your tokenizer. Marks will be awarded both for a reasonable overall classification performance, and a good trade-off between classification performance and efficiency (i.e., using fewer parameters). 10

- (e) Explain your tokenizer function and discuss its performance (text answer, 500 words maximum). 7
- (f) Five marks are allocated for your git commit history. You should make regular, atomic commits with concise but informative commit messages. See the section below titled **Submission** for more details. 5

Part Two total marks: 45

3. Part Three — Zero-shot and Few-shot Classification with LLMs

In this part of the coursework, you should perform the same political party classification task as in Part Two, but using prompting with a large language model instead of the scikit-learn classifiers. For this part use the smaller sample of the speech data in `hansard500.csv`. You may choose any open-weight model that is available to you through Openrouter, Ollama, or Huggingface. We will explore these models in class in Week 7. Use the same label set as in Part Two. Unless a question says otherwise, you should use the same train/test split as in Part Two.

- (a) State which model you used, whether you accessed it through Ollama or OpenRouter, and any generation parameters that you set. Briefly explain why you chose these settings. 4
- (b) Implement a zero-shot classification script for the party prediction task. Your prompt should instruct the model to output one label only. Report the exact zero-shot prompt that you used, and print the macro-average f1 score and classification report on the test set. 8
- (c) Implement few-shot classification for the same task. Construct a prompt that includes a small number of labelled examples from the training data. Report the exact few-shot prompt that you used, explain how you selected the examples, and print the macro-average f1 score and classification report on the test set. 8
- (d) Compare the zero-shot and few-shot results. Which performed better? How did you optimize your prompts? (Text answer, 500 words maximum). 5

Part Three total marks: 25

Submission

Submit to Moodle and confirm on Moodle

Submission is by uploading a zip file containing your python code, text files, and git commit history on Moodle.

Academic Declaration

You must also enter the following Academic Declaration into `README.md` for your submission:

“I have read and understood the sections of plagiarism in the College Policy on assessment offences and confirm that the work is my own, with the work of others clearly acknowledged. I give my permission to submit my report to the plagiarism testing database that the College is using and test it using plagiarism detection software, search engines or meta-searching software.”

This refers to the document at:

<https://www.bbk.ac.uk/student-services/exams/plagiarism-guidelines>

A submission without the declaration will get 0 marks.

Deadlines

- The submission deadline is: 26th June 2025, 14:00 UK time.
- The late cut-off deadline for receiving a capped mark is: 10th July 2025, 14:00 UK time.